overriding earlier ones; within each layer, standard specificity rules still apply.

- Scoped styles (`@scope`) do not affect specificity, except when the `:scope` pseudo-class is used, which does add to specificity.
- Inline styles can only be overridden by another style with `!important`, not by selectors alone.
- The `!important` declaration overrides normal specificity rules and should be used sparingly.
- If multiple `!important` declarations exist for the same property on the same element, the browser applies normal specificity rules among them. If specificity is equal, the last `!important` declaration in the stylesheet (by order) wins. If specificity is not equal, the `!important` declaration with higher specificity wins.

During one of Jane's frontend interviews, she was given the style definition shown in listing 2.6 and asked to determine the final color of the button, to test her CSS specificity knowledge.

Listing 2.6 CSS specificity example

```
const styles = `
  .button {
    padding: 10px 20px;
    font-size: 16px;
    cursor: pointer;
  }
  .primary-button {
    background-color: blue;
    color: white;
  }
  button {
    background-color: red;
    color: black;
  }
  .container .primary-button {
    background-color: green;
  }
  #my-button {
    color: yellow;
  }
`;
interface ButtonProps {
```